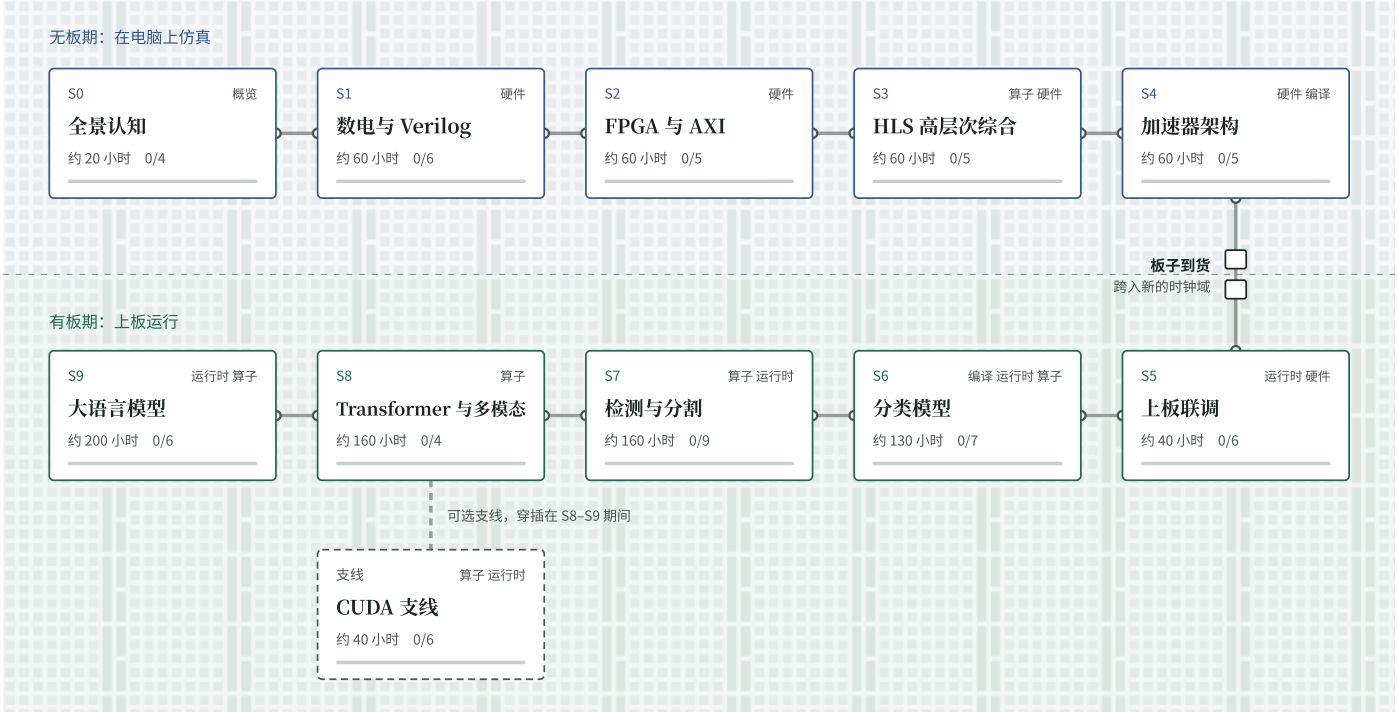
从门电路到大模型

FPGA 部署 AI 模型的学习路线

为没有硬件基础的你设计：先在电脑上用仿真把基础打牢，板子到货后依次部署分类、检测、多模态和语言模型。 量化不是这条路线的重点：先用最常见的 INT8 量化自己跑通，重心放在把量化后的模型变成在 FPGA 上运行的电路。



起点

本科学过部分数电、模电，没有 HDL 和 FPGA 经验。默认你熟悉 Python 和 PyTorch。

终点

能把量化稀疏后的分类、检测、多模态和语言模型部署到 FPGA，并能自己设计加速器。

分工

量化同学的进度不确定，路线不等他：前期用公开工具自己做最常见的 INT8 量化，按一份默认格式约定推进。他的成果到了再替换、对比。

工作量

S0	S1	S2	S3	S4	S5	S6	S7	S8	S9
20h	60h	60h	60h	60h	40h	130h	160h	160h	200h

整条主线预计约 **950 小时**。其中前五个阶段不需要板子，约 260 小时；上板之后的五个阶段约 690 小时。可选的 CUDA 支线另需约 40 小时，不计入主线。按已勾选的任务估算，目前完成约 **0 小时**。

小时数是按零基础估算的，误差可能有三到五成，做完 S1 后可以按实际用时校准。 时间零碎时这样安排：只有 1 小时，就做标着“1 小时可做”的任务，比如刷题、读论文、做统计； 有半天的时候，留给装工具、跑综合、上板调试这类一旦中断就要重新进入状态的任务。 每次结束前在阶段笔记里写一句“下次从哪继续”，下次能省下十几分钟的回想时间。 资料上，S1、S2 优先用标着“中文”的资料降低门槛，S3 以后逐步过渡到英文原版和论文。

阶段 0

全景认知与路线选型

约 20 小时　全部可在仿真中完成　概览

搞清楚“把模型放到 FPGA 上”有哪几条技术路线，各自适合哪个阶段。

FPGA 部署 AI 的三条路线差别很大。先想清楚组合方式，后面就不会在工具链上反复试错。

要掌握的概念

- **FPGA 的定位**：可重构的空间计算。优势是低延迟、低功耗、位宽可定制、对稀疏友好；劣势是开发周期长，主频只有 200–400 MHz。
- **路线 A 厂商 AI 栈**：AMD Vitis AI + DPU。DPU 是现成的加速器 IP，编译模型就能跑。上手快，但内部是黑盒，自定义的稀疏格式基本用不上。
- **路线 B 开源编译框架**：hls4ml、FINN。把量化模型自动生成为数据流硬件，适合小模型和学习原理。
- **路线 C 自研加速器**：用 HLS 或 RTL 写计算阵列。只有这条路能完整吃到稀疏和低比特的收益，是研究的主战场，也最难。
- **建议组合**：A 做性能基线、理解完整系统；B 用来学习和快速验证；C 是最终要掌握的能力。

动手任务

- [ ] 读 Sze 等《Efficient Processing of Deep Neural Networks》第 1–2 章，或看 MIT 6.5930 前两讲　1 小时可做　仿真
- [ ] 浏览 Vitis AI、FINN、hls4ml 的文档首页和示例，每个用三句话写下它适合做什么　1 小时可做　仿真
- [ ] 问一下量化同学的进度和打算用的格式。路线不等他，前期按最常见的 INT8 自己量化　1 小时可做　仿真
- [ ] 和导师确认目标档位（会用工具链部署 / 自研加速器到 CNN / 一直做到 LLM），以及实验室有哪几块板卡　1 小时可做　仿真

> **验收标准**
> 一页纸：三条路线的对比，加上你打算采用的组合。

学习资料

- Efficient Processing of Deep Neural Networks（Sze, Chen, Yang, Emer, 2020）
  DNN 硬件加速的标准教材
- MIT 6.5930 Hardware Architectures for Deep Learning
  课程讲义与视频公开
- Vitis AI 文档 https://xilinx.github.io/Vitis-AI/
  看支持的板卡和模型即可

容易踩的坑

- 一上来就找“一键把 LLM 部署到 FPGA”的教程。没有通用方案，LLM 是这条路线的最后一站。

待讨论

- 最终产出偏论文研究（重路线 C），还是偏工程落地（重路线 A）？
- 实验室是否已指定厂商（AMD/Xilinx 还是 Intel/Altera）或具体板卡？

> **笔记：**
> 两者都侧重，如果后期做论文就会侧重软硬件协同。
> 实验室应该是做AMD方向的板卡。

阶段 1

数字电路回炉 + Verilog

约 60 小时　　全部可在仿真中完成　　硬件

能用 Verilog 写出组合电路、时序电路和状态机，跑仿真并看懂波形。

后面即使主要写 HLS，调试、读综合报告、写接口胶水逻辑也离不开 RTL 思维。这一阶段要建立的直觉是：硬件是同时工作的电路，不是逐行执行的程序。

要掌握的概念

- **组合逻辑与时序逻辑**：触发器、寄存器、时钟、复位。
- **阻塞赋值与非阻塞赋值**：写错是新手最常见的仿真/综合不一致来源。
- **有限状态机**：控制逻辑的标准写法。
- **时序**：建立/保持时间、关键路径、最高频率，以及用流水线提高频率。
- **定点数**：补码、Q 格式、乘加器（MAC）与累加位宽。
- **握手协议**：valid/ready，后面 AXI-Stream 的基础。

动手任务

- [ ] 刷 HDLBits 到 Sequential Logic 章节结束（约 120 题）　1 小时可做　仿真
- [ ] 本地装 Icarus Verilog + GTKWave（或 Verilator），跑通第一个 testbench　1 小时可做　仿真
- [ ] 写一个带 valid/ready 握手的同步 FIFO 并仿真　1 小时可做　仿真
- [ ] 写 INT8 乘加单元（32 位累加），用 Python 生成随机数据比对结果　1 小时可做　仿真
- [ ] 用 NVBoard 虚拟开发板做一个带数码管显示的计数器，在电脑上看到拨开关、亮灯的效果　仿真
- [ ] 进阶：写 4×4 脉动阵列做矩阵乘，用 cocotb（Python 写 testbench）验证　仿真

> **验收标准**
> 4×4 脉动阵列 GEMM 的 RTL 和自动比对的 testbench，能对着波形讲清楚数据怎么流动。

学习资料

- HDLBits https://hdlbits.01xz.net
  在线刷题，即时判题，最适合零基础
- HDLBits 中文版电子书 https://github.com/OpenIC-SIG/HDLBits-Chinese-gitbook 中文
  HDLBits 题目的中文翻译，和原站对照着刷
- Harris & Harris《数字设计和计算机体系结构》第 1–5 章 中文
  补数电基础，用中文版即可
- 一生一芯（只做前期内容） https://ysyx.oscc.cc 中文
  做前期的 Linux、C、数字电路和 Verilator 部分；后面的处理器设计和流片与本路线关系不大
- NVBoard 虚拟 FPGA 开发板 https://github.com/NJU-ProjectN/nvboard 中文
  配合 Verilator 在电脑上仿真开关、LED、数码管，没板子也能看到电路效果
- cocotb https://www.cocotb.org
  用 Python 写 testbench，适合有 AI 背景的人
- Nandland https://nandland.com
  FPGA 入门讲解清楚

容易踩的坑

- 把 Verilog 当 C 写：for 循环会被展开成一堆硬件，而不是循环执行。
- 只看仿真打印的结果，不看波形。
- 忽略复位和时钟，仿真能过、上板出错。

待讨论

- 你的 Python / PyTorch 熟练度如何？这决定 testbench 主要用 cocotb 还是纯 Verilog。

阶段 2

FPGA 架构、Vivado 流程与 AXI 总线

约 60 小时　全部可在仿真中完成　**硬件**

理解 FPGA 内部资源，能把设计从 RTL 跑到比特流，看懂资源和时序报告，掌握 AXI 总线。

板子还没到也能把整套工具流程跑到生成比特流。资源报告会告诉你一块板子到底能放多大的加速器。

要掌握的概念

- **LUT、FF、CLB**：通用逻辑资源。
- **DSP48**：乘加的主力，数量基本决定了算力上限。
- **BRAM / URAM**：片上存储，决定能缓存多少权重和激活。
- **布线与时序收敛**：WNS 为负说明时序没满足。
- **Zynq 架构**：PS（ARM 处理器）+ PL（可编程逻辑），AI 部署最常用的形态。
- AXI4 搬数据、AXI4-Lite 配寄存器、AXI4-Stream 传数据流；DMA；DDR 带宽。

动手任务

- [ ] 在 Linux 上安装 Vivado / Vitis（免费 Standard 版即可，预留约 150 GB 磁盘）　仿真
- [ ] 把 S1 的脉动阵列综合实现到目标器件（如 KV260 的 XCK26），读资源利用率和时序报告　仿真
- [ ] 把阵列逐步加大，记录 DSP 用量和最高频率的变化　1 小时可做　仿真
- [ ] 用 Block Design 搭 Zynq PS + 自定义 AXI-Lite IP 的系统，生成比特流　仿真
- [ ] 用 AXI VIP 仿真验证 AXI-Lite 寄存器读写　仿真

> **验收标准**
> 一份报告：你的阵列在目标器件上能放多大、跑多快，瓶颈是 DSP、BRAM 还是时序。

学习资料

- 国内开发板厂商的 Vivado 视频教程（正点原子、ALINX 黑金、米联客等）　中文
  B 站可搜到。Vivado 界面操作讲得细，当操作手册用；AI 加速部分讲得浅
- AMD UG1037 AXI Reference Guide
  AXI 协议的实用说明
- AMD UG1085 Zynq UltraScale+ 技术参考手册
  查阅用，不必通读
- The Zynq Book
  免费电子书，基于 Zynq-7000，概念通用

容易踩的坑

- 随手装最新版。Vivado、Vitis AI、PetaLinux 和板卡 BSP 的版本必须匹配，先查兼容表再装。
- 时序违例的比特流上板后行为不可预测，别抱侥幸。

待讨论

- 服务器系统版本是否在 Vivado 支持列表内？有没有图形界面（Vivado GUI 需要 X11 或 VNC）？

阶段 3

HLS 高层次综合

约 60 小时　全部可在仿真中完成　算子　硬件

用 C++ 写加速器，通过 pragma 控制并行度，独立完成 GEMM 和卷积 kernel 的优化。

HLS 是做 AI 加速器效率最高的入口，hls4ml 和 FINN 底层也是 HLS。你有软件背景，这一阶段会比 RTL 上手快。

要掌握的概念

- **流程**：C 仿真（csim）→ 综合 → 联合仿真（cosim）→ 导出 IP。
- **PIPELINE 与 II**：II=1 表示每个周期启动一次新迭代。
- UNROLL 与 ARRAY_PARTITION：展开计算的同时要把存储切开，否则访存会卡住并行度。
- **DATAFLOW**：任务级流水，多个函数同时工作。
- **任意精度类型**：ap_int、ap_fixed，直接对应量化后的位宽。
- **接口**：m_axi 访问 DDR，s_axilite 配参数，axis 走数据流。
- 循环分块（tiling）与双缓冲（ping-pong）：大模型放不进片上存储时的标准做法。

动手任务

- [ ] 跑通 Vitis-HLS-Introductory-Examples 里的 pipeline、array partition、dataflow 示例　1 小时可做　仿真
- [ ] 写 INT8 GEMM：从朴素三重循环开始，逐步优化到 II=1，每一步记录延迟和资源　1 小时可做　仿真
- [ ] 写 3×3 卷积 kernel（line buffer + 滑窗），通道数可配置　仿真
- [ ] 给 GEMM 加上 m_axi 接口和分块，模拟从 DDR 搬数据　仿真
- [ ] 实现一种稀疏 GEMM（如 2:4 结构化稀疏），和稠密版本对比资源与周期　仿真

> **验收标准**
> 一张优化表：同一个 GEMM 在不同 pragma 下的周期数、DSP/BRAM 用量和频率。

学习资料

- Kastner 等《Parallel Programming for FPGAs》 https://arxiv.org/abs/1805.03648
  免费，HLS 入门最好的教材
- pp4fpgas-cn：上一本书的中文翻译 https://xupsh.gitbook.io/pp4fpgas-cn/　中文
  配套 HLS 工程在 GitHub 的 xupsh/pp4fpgas-cn-hls。译稿不一定跟上原书最新版，有疑问时对照英文
- AMD UG1399 Vitis HLS 用户指南
  pragma 的权威说明
- Xilinx/Vitis-HLS-Introductory-Examples https://github.com/Xilinx/Vitis-HLS-Introductory-Examples
  小而全的示例集
- Xilinx/Vitis-Tutorials https://github.com/Xilinx/Vitis-Tutorials
  官方教程合集

容易踩的坑

- 用了动态内存、递归、STL 容器，这些不可综合。
- csim 通过就以为对了。必须过 cosim。
- 盲目 UNROLL，资源直接爆掉。

待讨论

阶段 4

AI 加速器体系结构（可与 S3 并行）

约 60 小时　全部可在仿真中完成　硬件　编译

能估算一个模型在某块板子上的性能上限，并设计合理的计算阵列和存储层次。

这一阶段区分“会用工具”和“会设计加速器”。稀疏和低比特的收益能不能兑现，取决于这里的设计。

要掌握的概念

- **Roofline 模型**：算力上限、带宽上限和算术强度，判断一层是算力受限还是带宽受限。
- **数据复用方式**：权重固定、输出固定、行固定（Eyeriss）。
- **两种架构**：逐层流水的数据流架构（FINN）与通用覆盖层（DPU 一类 overlay）。
- 片上缓冲规划与 tiling 参数搜索。
- **量化算子**：INT8×INT8→INT32 累加，再用乘法加移位做 requantize。
- **量化的选择**：INT8 最常见，先用它跑通；之后可以试 INT4 或混合精度、FP8 或 MX 块浮点、2 的幂量化。位宽越低，一个 DSP 能做的乘法越多，权重带宽也越省。
- **稀疏**：非结构化稀疏很难在硬件上兑现加速，N:M 和块稀疏更友好。
- **压缩格式**：CSR、CSC、bitmask，以及索引带来的额外存储和带宽开销。

动手任务

- [ ] 用 Python 脚本统计 ResNet-18 每层的 MAC 数、参数量和激活大小　1 小时可做　仿真
- [ ] 以 KV260 为目标（DSP 数、片上存储、DDR 带宽）画 Roofline，标出每层的位置　1 小时可做　仿真
- [ ] 精读 Zhang et al. FPGA'15 和 Eyeriss，用自己的话复述 tiling 和数据流设计　1 小时可做　仿真
- [ ] 读 EIE，搞清楚稀疏压缩格式在硬件上怎么解码　1 小时可做　仿真
- [ ] 按页面下方的“量化与数据格式约定”定下自己的默认格式（先全部用默认值）　1 小时可做　仿真

> **验收标准**
> 一份加速器设计文档：目标板、阵列规模、存储规划、各层预估延迟。这是上板前的最后一个里程碑。

学习资料

- Zhang et al., Optimizing FPGA-based Accelerator Design for Deep CNNs, FPGA 2015
  Roofline + tiling 的经典范例
- Chen et al., Eyeriss, ISCA 2016
  数据复用与行固定数据流
- Han et al., EIE, ISCA 2016
  稀疏压缩模型的硬件推理
- Williams et al., Roofline, CACM 2009
  性能上限分析方法

容易踩的坑

- 只算峰值算力，忽略带宽。
- 稀疏率 90% 不等于加速 10 倍，索引开销和负载不均会吃掉大部分收益。

待讨论

- S6 自己做 INT8 量化时，用 PyTorch 自带的量化工具还是 Vitis AI 量化器？前者通用、方便导出给自研加速器，后者的结果能直接给 DPU 跑。

阶段 5

上板 Bring-up

约 40 小时　需要板子，其中 0 项任务可先仿真　运行时　硬件

打通“ARM 端软件 ↔ FPGA 加速器”的完整数据通路。

仿真里正确的设计，上板后还要过系统、驱动、内存一致性这几关。先用最小的设计把通路打通，再上模型。

要掌握的概念

- **启动流程**：ARM 上跑 Linux，运行时动态加载比特流。
- **PYNQ**：用 Python 直接操作硬件，最适合入门。
- **Vitis 嵌入式流程与 XRT**：更正式的软件栈；PetaLinux 定制系统可以后面再学。
- **嵌入式 Linux 基础**：交叉编译、设备树、驱动加载。Zynq 的 ARM 端跑的是 Linux，这部分比单片机知识更有用。
- **DMA 与 cache 一致性**：最常见的“结果偶尔不对”的原因。
- **ILA 片上逻辑分析仪**：在真实硬件上抓信号。

动手任务

- [ ] 按官方指南给板子烧录启动镜像（KV260 可选 Ubuntu 或 PYNQ）　需板
- [ ] 点灯：PL 侧 LED 闪烁，确认工具链到板卡全通　需板
- [ ] 把 S2 的 AXI-Lite IP 上板，从 Python 读写寄存器　需板
- [ ] 把 S3 的 HLS GEMM 上板，Python 喂数据，结果与 NumPy 比对　需板
- [ ] 测实际吞吐与理论值的差距，定位是计算、DMA 还是软件开销　需板
- [ ] 用 ILA 抓一次 AXI 传输波形　需板

> **验收标准**
> 一个 Jupyter Notebook：在板上调用自己写的 GEMM 加速器，给出正确性和实测带宽、吞吐。

学习资料

- PYNQ https://www.pynq.io
  Python 驱动 FPGA 的框架
- 开发板厂商的 Zynq 实操教程　中文
  板卡启动、PS 与 PL 交互的中文讲解。选和你板卡型号对应的那一套
- Xilinx/Kria-PYNQ https://github.com/Xilinx/Kria-PYNQ
  KV260 上安装 PYNQ
- Kria KV260 官方入门文档
  启动镜像与开箱流程

容易踩的坑

- 没有刷 cache，读到旧数据。
- 比特流、设备树和驱动版本不匹配。

待讨论

- 板子的系统镜像由谁维护？能否远程访问（串口、网络）？

阶段 6

分类模型

约 130 小时　　需要板子，其中 4 项任务可先仿真　　编译　运行时　算子

在 FPGA 上跑通分类模型。三条路线各走一遍，最终用自己的加速器跑量化稀疏版 ResNet。

分类模型结构规整，是验证整条部署链路最好的起点。这里搭好的逐层比对和测量方法，后面每一站都会复用。

要掌握的概念

- **端到端流程**：量化模型 → 图解析（ONNX / QONNX）→ 算子映射 → 权重打包 → 硬件执行 → 后处理。
- **层融合**：conv + bn + relu 合并成一个硬件算子。
- **残差连接**：需要额外缓存一路激活。
- **验收指标**：Top-1 精度损失、延迟、FPS、功耗、FPS/W、资源利用率。

动手任务

- [ ] 用 PyTorch 自带量化工具把 ResNet-18 做 INT8 训练后量化，导出整数权重、scale 和每层 golden 输出　仿真
- [ ] 用 hls4ml 把 MNIST MLP 转成 HLS 工程，完成 csim 和 cosim　仿真
- [ ] 跑通 FINN 官方的 CIFAR-10 端到端示例（CNV 网络）　仿真
- [ ] 用自写加速器跑 LeNet 或小 CNN，逐层与 Python golden 比对　仿真
- [ ] 用 Vitis AI + DPU 在板上跑 ResNet-50，作为性能基线　需板
- [ ] 用自写加速器跑 INT8 ResNet-18，再用幅值剪枝做一版 2:4 稀疏，对比稠密和稀疏的实测延迟（量化同学的模型到了再替换）　需板
- [ ] 测功耗（板载传感器或功率计），算出能效　1 小时可做　需板

> **验收标准**
>
> 一张对比表：DPU、FINN、自研加速器在同一模型上的精度、延迟、能效和资源；自研加速器一栏包含 INT8 稠密和 2:4 稀疏两个版本。

学习资料

- hls4ml https://fastmachinelearning.org/hls4ml/
  Keras / PyTorch / QONNX 转 HLS
- FINN https://xilinx.github.io/finn/
  配合 Brevitas 量化训练的数据流编译器
- Umuroglu et al., FINN, FPGA 2017
  数据流架构的代表论文
- Vitis AI Model Zoo
  现成的量化模型，做基线用
- PyTorch 量化文档 https://pytorch.org/docs/stable/quantization.html
  训练后量化（PTQ）的标准做法，自己量化模型时用

容易踩的坑

- 不做逐层 golden 比对，最后输出错了无从查起。
- Vitis AI 对板卡和 Vivado 版本要求严格，开工前先查兼容表。

待讨论

- Vitis AI DPU 只作为基线，还是也考虑作为最终方案的一部分？

阶段 7

检测与分割模型

约 160 小时　需要板子，其中 4 项任务可先仿真　**算子**　运行时

部署轻量 YOLO 检测模型，完成从摄像头输入到画框输出的实时系统；再部署语义分割和实例分割模型。

检测比分类多出大分辨率输入、多尺度输出、上采样和拼接算子，以及复杂的后处理。分割又多一个解码器：要把特征图放大回原图分辨率，编码器的中间结果要一直存到解码器用到为止，输出是逐像素的。两者合起来，考验的是软硬件划分和存储带宽规划。

要掌握的概念

- **轻量检测结构**：YOLOv5n、YOLOv8n 一类，以及其中的上采样、concat、多分支。
- **检测后处理**：框解码和 NMS 一般放在 ARM 端。
- **编码器-解码器与跳跃连接**：UNet 一类结构里，编码器每一级的输出要缓存到解码器对应层，片上放不下就得进出 DDR。
- **上采样算子**：转置卷积、双线性插值、最近邻插值，硬件实现代价差别很大。
- **空洞卷积**：DeepLab 系列常用，访存模式和普通卷积不同。
- **实例分割**：YOLOv8-seg 输出 mask 原型和系数，mask 合成可以放 ARM 端，也可以做成一个小矩阵乘放进 FPGA。
- **软硬件流水**：预处理、推理、后处理三段重叠执行。
- **视频通路**：MIPI 摄像头 → 预处理 → 推理 → 显示。
- **量化敏感度**：INT8 对 mAP 和 mIoU 的影响通常比对分类精度更明显，分割的边缘细节尤其容易受影响。

动手任务

- [ ] 统计 YOLO 小模型各层计算量，用 S4 的方法评估能否放下　1 小时可做　仿真
- [ ] 扩展自研加速器：支持上采样、concat、stride-2 卷积　仿真
- [ ] 统计 UNet 各级跳跃连接需要缓存的激活大小，判断哪些能留在片上、哪些必须进 DDR　1 小时可做　仿真
- [ ] 扩展加速器：支持转置卷积或双线性上采样、空洞卷积，cosim 验证　仿真
- [ ] 用 Vitis AI 在板上跑官方 YOLO 示例，或体验 KV260 Smart Camera 应用　需板
- [ ] backbone 用自研加速器，后处理放 ARM，跑通单张图片检测　需板
- [ ] 接摄像头做实时检测，测端到端延迟和 FPS　需板
- [ ] 部署轻量语义分割模型（如小型 UNet，或 MobileNet 骨干的 DeepLabv3+），测 mIoU 和 FPS　需板
- [ ] 选做：部署 YOLOv8n-seg 实例分割，比较 mask 合成放 ARM 和放 FPGA 的延迟　需板

> **验收标准**
> 实时检测 demo 和语义分割 demo，各附一份延迟分解（预处理、推理、后处理），以及分割模型的 DDR 带宽占用分析。

学习资料

- KV260 Smart Camera 加速应用
  官方的检测完整系统，可当参考答案
- Vitis AI Model Zoo 中的检测与分割模型
  基线
- Ultralytics YOLO 文档 https://docs.ultralytics.com
  检测、实例分割模型的结构与 ONNX 导出
- Ronneberger et al., U-Net, MICCAI 2015
  编码器-解码器 + 跳跃连接的原型
- Chen et al., DeepLabv3+, ECCV 2018
  空洞卷积与轻量解码器

容易踩的坑

- 只优化推理，结果瓶颈在 Python 后处理或图像缩放。
- mAP 或 mIoU 下降找不到原因。需要逐层检查量化误差。
- 分割模型的跳跃连接激活反复进出 DDR，带宽成为瓶颈，计算阵列大部分时间在等数据。
- 高分辨率分割图的 argmax 和上色放在 Python 里做，比推理还慢。

待讨论

- 你们的应用更需要语义分割（每个像素分类）还是实例分割（区分每个物体）？输入分辨率大概多少？

阶段 8

Transformer 与多模态

约 160 小时　　需要板子，其中 2 项任务可先仿真　　算子

让加速器支持 Transformer 核心算子，先部署 ViT 小模型，再部署图文多模态模型的视觉编码部分。

从 CNN 到 Transformer，计算主体从卷积变成大矩阵乘，同时多出 softmax、LayerNorm、GELU 等非线性算子。这是通往 LLM 的必经之路。

要掌握的概念

- **自注意力**：QKV 投影、QK^T、softmax、乘 V。
- **非线性算子硬件化**：查表、分段线性近似、整数近似（I-BERT 的做法）。
- **LayerNorm**：要对整行求均值和方差，需要归约和额外缓存。
- **动态矩阵乘**：QK^T 的两个操作数都是激活，和固定权重的矩阵乘在数据流上不同。
- **token 剪枝与稀疏注意力**：Transformer 上的稀疏化手段。
- **多模态**：视觉编码器（ViT）+ 投影层 + 语言模型，可以先只加速视觉编码器。

动手任务

- [ ] 用 HLS 实现定点版 softmax、LayerNorm、GELU，测与浮点的误差　仿真
- [ ] 实现多头注意力模块，在 cosim 中验证　仿真
- [ ] 部署 DeiT-Tiny 或 ViT-Tiny 分类，与 S6 的 CNN 比较效率　需板
- [ ] 部署 CLIP 类模型的图像编码器，文本侧先放 CPU　需板

> **验收标准**
> ViT-Tiny 在板上运行，给出各算子的延迟占比，说明非线性算子的开销。

学习资料

- Kim et al., I-BERT: Integer-only BERT Quantization, ICML 2021
  整数化 softmax / GELU / LayerNorm
- Dong et al., HeatViT, HPCA 2023
  FPGA 上的 ViT token 剪枝加速
- Dosovitskiy et al., ViT, ICLR 2021
  视觉 Transformer 原文

容易踩的坑

- softmax 直接用浮点 exp，资源暴涨。
- 忽略注意力矩阵随序列长度平方增长的存储需求。

待讨论

阶段 9

大语言模型

约 200 小时　　需要板子，其中 3 项任务可先仿真　　运行时　算子

在 FPGA 上运行一个 0.5B–1.5B 量级小型 LLM 的完整推理，理解 LLM 推理为什么主要受带宽限制。

解码阶段每生成一个 token 都要把全部权重读一遍，瓶颈是内存带宽而不是算力。这恰好是稀疏和低比特量化在 FPGA 上最能发挥价值的地方。

要掌握的概念

- **Prefill 与 Decode**：前者计算受限，后者带宽受限，最好用不同的硬件策略。
- **KV Cache**：随生成长度增长，需要专门的存储管理。
- **混合精度**：权重 INT4、激活 INT8，group-wise 量化。
- **稀疏权重**：直接减少要搬运的字节数。
- **存储**：HBM 与 DDR 的带宽差一个数量级。
- **主机协同**：PCIe、XRT；采样和 tokenizer 放在 CPU 端。

动手任务

- [ ] 用 Python 估算：0.5B 模型 INT4 权重在 KV260（DDR4 约 19 GB/s）和 HBM 卡（约 460 GB/s）上的解码速度上限　1 小时可做　仿真
- [ ] 先用 llama.cpp 或 PyTorch 在 CPU 上跑通同一模型，作为功能和性能 golden　仿真
- [ ] 实现带 KV Cache 的 decoder layer，cosim 验证单层　仿真
- [ ] 先在小板上跑通极小模型（几十 M 参数的玩具 GPT）　需板
- [ ] 在 HBM 板卡或 Versal 上部署 0.5B–1.5B 模型，测 tokens/s 和能效　需板
- [ ] 加入稀疏权重解码，看带宽节省是否转化为速度提升　需板

> **验收标准**
> 端到端 LLM 推理 demo，加一份带宽利用率分析报告。

学习资料

- Zeng et al., FlightLLM, FPGA 2024
  稀疏 + 混合精度的 FPGA LLM 推理，和你的方向最接近
- Hong et al., DFX, MICRO 2022
  多 FPGA 的 GPT 推理
- Chen et al., Understanding the Potential of FPGA-Based Spatial Acceleration for LLM Inference, TRETS 2024
  LLM 在 FPGA 上的性能建模
- Allo https://github.com/cornell-zhang/allo
  上一篇论文配套的加速器设计语言

容易踩的坑

- 在 KV260 这类边缘板上期待大模型。4 GB 内存和约 19 GB/s 带宽决定了上限。
- prefill 和 decode 用同一套硬件策略。

待讨论

- LLM 阶段能否申请到 HBM 板卡（如 Alveo U280 / U55C）、Versal 平台或云 FPGA 实例?

可选支线：GPU 与 AI infra 衔接

约 40 小时　需要 NVIDIA GPU，不需要 FPGA 板；建议穿插在 S8–S9 期间　算子　运行时

能写基本的 CUDA 和 Triton kernel，读懂主流推理引擎的核心机制，并能为论文跑出调优过的 GPU 基线。

FPGA 加速器论文几乎都要和 GPU 对比，GPU 基线没调好，结论就站不住。这些也是 AI infra 岗位的核心技能，而且和你在 FPGA 上学的 tiling、算子融合、量化一一对应，迁移成本很低。

要掌握的概念

- **GPU 执行模型**：线程、warp、block、SM。对照 FPGA：GPU 靠大量线程分时复用硬件，FPGA 靠把电路空间展开。
- **存储层次**：寄存器、shared memory、L2、HBM。shared memory 分块和你在 FPGA 上的 BRAM 分块是同一个思路。
- **Tensor Core 与低精度**：INT8、FP8，以及 Ampere 架构开始硬件支持的 2:4 结构化稀疏，可以直接和你的 FPGA 稀疏方案对比。
- **Triton**：用 Python 写 GPU kernel，比 CUDA 上手快，适合快速验证想法。
- **FlashAttention**：分块计算加在线 softmax，避免把整个注意力矩阵写回显存。
- **推理引擎机制**：llama.cpp 的量化格式和反量化 kernel；vLLM 的 PagedAttention 和连续批处理。
- **性能分析**：Nsight Systems 看整体时间线，Nsight Compute 看单个 kernel 的带宽和算力利用率。

动手任务

- [ ] 读 FlashAttention 论文，把它的分块思路和你在 S8 做的 FPGA 注意力实现对照　1 小时可做　只需电脑
- [ ] 读 llama.cpp 的量化格式和一个反量化 kernel 的实现　1 小时可做　只需电脑
- [ ] 写 CUDA 矩阵乘：从朴素版到 shared memory 分块，用 Nsight Compute 看带宽利用率　需 GPU
- [ ] 跟着 Triton 官方教程写 GEMM 和融合 softmax，和 CUDA 版本对比性能　需 GPU
- [ ] 用 vLLM 或 TensorRT-LLM 跑和 S9 相同的模型，测 tokens/s 和能效，作为 FPGA 方案的 GPU 基线　需 GPU
- [ ] 用 PyTorch 的 2:4 半结构化稀疏在 GPU 上跑一次推理，和你的 FPGA 稀疏方案对比　需 GPU

> **验收标准**
> 一张 GPU 与 FPGA 的对比表：同一模型的延迟、吞吐、功耗、能效，GPU 一侧附带 profiling 结果，证明基线已经调优。

学习资料

- Kirk & Hwu《Programming Massively Parallel Processors》
  CUDA 入门的标准教材，有中文版
- NVIDIA CUDA C++ Programming Guide
  查阅用
- Triton 官方教程 https://triton-lang.org
  矩阵乘和融合 softmax 两篇最值得做
- Dao et al., FlashAttention, NeurIPS 2022
  分块思想在 GPU 上的代表作
- Kwon et al., vLLM / PagedAttention, SOSP 2023
  LLM 推理引擎的 KV Cache 管理
- llama.cpp https://github.com/ggml-org/llama.cpp
  量化格式和 CPU/GPU kernel 都值得读

容易踩的坑

- 拿没调优的 PyTorch eager 模式结果当 GPU 基线，审稿人会质疑对比不公平。
- 只比吞吐，不比功耗和批大小。FPGA 的优势通常在 batch=1 的低延迟和能效。
- 在支线上投入太多，挤占主线时间。按每周 7 小时，这条支线控制在 30–50 小时以内。

待讨论

- 实验室有没有可用的 NVIDIA GPU？型号是什么？2:4 稀疏需要 Ampere（如 A100、RTX 30 系）及以后的架构。

这条路线在 AI 系统栈中的位置

AI infra 自上而下大致分六层。这条路线以最底下的硬件层为主体，向上覆盖算子、编译和运行时三层，可以算作 FPGA 版的 AI infra 下半部分。 每个阶段标题下的层次标签对应这里的分层，第一个是主要层次。下面每层列出的阶段里，加粗的是以该层为主的阶段。

集群与调度
不涉及

不涉及。
GPU 世界的对应：Kubernetes、训练集群调度

分布式训练
不涉及

不涉及。这条路线做推理，不做训练。
GPU 世界的对应：Megatron-LM、DeepSpeed、NCCL

推理引擎与运行时
部分覆盖

主机和加速器协同、前后处理流水、DMA 调度、KV Cache、prefill 与 decode 的调度。
GPU 世界的对应：vLLM、TensorRT-LLM、llama.cpp

S5 上板联调 S6 分类模型 S7 检测与分割 **S9 大语言模型** CUDA 支线

AI 编译器
部分覆盖

把模型图映射到硬件：图解析、算子映射、层融合、tiling 参数、权重打包。hls4ml、FINN、Allo 都属于这一层。
GPU 世界的对应：TVM、MLIR、Triton

S4 加速器架构 **S6 分类模型**

算子与 kernel
覆盖

单个算子的高效实现。GPU 上写 kernel，你在 FPGA 上用 HLS 写电路。
GPU 世界的对应：CUDA kernel、FlashAttention

S3 HLS 高层次综合 S6 分类模型 **S7 检测与分割**
S8 Transformer 与多模态 S9 大语言模型 **CUDA 支线**

硬件与加速器架构
覆盖

计算阵列、片上存储层次、数据流设计，学术上属于计算机体系结构。这是路线的主体。
GPU 世界的对应：GPU、TPU、各类 AI 芯片

S1 数电与 Verilog **S2 FPGA 与 AXI** S3 HLS 高层次综合 **S4 加速器架构**
S5 上板联调

板卡怎么选

板子到货前不必着急，前五个阶段和后面阶段里标着“仿真”的任务都不需要板子。 如果只能先买一块，选 KV260；LLM 阶段再另外申请 HBM 板卡。价格随渠道波动，表中只给量级。

板卡	芯片	关键资源	价格	覆盖阶段	说明
PYNQ-Z2	Zynq-7020	220 DSP，512 MB DDR3	千元级	S1–S5、FINN 小模型	便宜，Python 生态好；资源太小，放不下 YOLO 及以后的模型。
Kria KV260 推荐先买	Zynq UltraScale+（K26）	1248 DSP，4 GB DDR4，MIPI 摄像头接口	两三千元	S5–S8（ViT 小模型）	首选。Vitis AI、PYNQ、FINN 都支持，有官方检测应用可参考。
ZCU104 / ZCU102	Zynq UltraScale+	1728 / 2520 DSP	万元级	S6–S8，资源更宽裕	资料多，常见于论文实验；性价比不如 KV260。
Alveo U55C / U280	Virtex UltraScale+ HBM	9024 DSP，HBM2 约 460 GB/s，PCIe 插卡	数万元	S9 大语言模型	LLM 解码需要的高带宽。插在服务器上用，不是独立开发板。
Versal VEK280	Versal AI Edge（AIE-ML）	AI Engine 阵列 + 可编程逻辑	数万元	S8–S9	AI Engine 的编程模型和 HLS 不同，需要额外学习成本。

硬件周边技能要不要学

FPGA 和嵌入式领域的技能很多，但你的目标是部署 AI 模型。判断标准只有一个：它会不会出现在“模型 → 加速器 → 板上运行”这条链路上。

要学

FPGA 的数据通路：DMA、AXI-Stream、跨时钟域、视频输入

AI 部署的瓶颈经常不在计算，而在数据怎么进出 FPGA。这部分是传统 FPGA 工程里和你最相关的，已经放进 S2、S5、S7，不用另开一条线。

按项目再学

高速数据采集：ADC 接口、LVDS、JESD204、千兆网或 PCIe 采集卡

只有应用需要从传感器直接采数据再推理时才用得上，比如雷达、工业检测、医疗信号。等项目确定再补，有 S2 的基础大约 2–4 周能上手。纯摄像头图像输入用不到。

按项目再学

传统数字信号处理：FIR、FFT、DDS 等

模型输入是雷达、音频、振动这类一维信号时，把预处理放在 FPGA 上很有价值；做图像就跳过。图像预处理（缩放、颜色转换）已经包含在 S7 里。

不用学

STM32 等单片机

Zynq 的 ARM 端跑 Linux，你需要的是嵌入式 Linux（交叉编译、设备树、驱动），已经加进 S5。单片机裸机开发和单片机上跑 AI（TinyML）是另一条路线，和 FPGA 加速器的知识几乎不复用。

不用学，会看原理图就够

PCB 画板

开发板和核心板完全够用，KV260 本身就是核心板加载板。DDR、高速串行这类高速板设计是专门的硬件工程，和你的目标无关。你只需要读懂板卡用户手册和原理图：引脚约束、时钟从哪来、外设接在哪，一两天就够。将来真要做定制载板，找硬件工程师合作。

不用补

模拟电路

只在画板和设计 ADC 前端时才用得上。数字设计需要的电气常识（时钟、电平、上下拉），S1 和看原理图时顺带就会了。

量化与数据格式约定

量化结果的格式直接决定硬件怎么设计。每一项都先按最常见的默认做法来，自己用公开工具完成量化，不等量化同学； 跑通之后再尝试后面列的方向，同学的成果到了再对照替换。勾选表示这一项已经按默认值落实。

- [ ] **位宽** 默认：权重和激活都用 INT8，累加器 INT32。
  之后可试：W4A8、逐层混合精度

- [ ] **量化方式** 默认：对称量化；权重按输出通道（per-channel），激活按整个张量（per-tensor）。
  之后可试：LLM 权重按 group 量化，group 大小 64 或 128

- [ ] **scale 的表示** 默认：量化工具给出浮点 scale，部署时换算成定点乘数加右移位数，硬件里只做整数运算。
  之后可试：2 的幂 scale，乘法直接变移位

- [ ] **舍入与溢出** 默认：和所用量化工具的舍入方式保持一致；溢出时饱和到 [-128, 127]。

- [ ] **稀疏模式** 默认：第一版不做稀疏；第二版用 2:4 结构化稀疏（每 4 个权重里保留 2 个）。
  之后可试：块稀疏、更高稀疏率的 N:M

- [ ] **存储格式** 默认：稠密权重按 AXI 总线位宽对齐打包；2:4 稀疏每组存 2 个值加 2 个 2 位索引。
  之后可试：bitmask、CSR 等通用稀疏格式

- [ ] **非线性层** 默认：CNN 全部整数化；Transformer 的 softmax、LayerNorm 先用 FP16 或定点，跑通后再整数化。
  之后可试：I-BERT 式全整数近似

- [ ] **导出格式** 默认：ONNX（QDQ 格式）加权重二进制文件，再加一个描述每层 scale 和形状的 JSON。

- [ ] **逐层比对数据** 默认：用 PyTorch 导出每层整数域的输入和输出，作为硬件逐层比对的 golden。

- [ ] **精度验收线** 默认：INT8 分类模型 Top-1 相对浮点下降不超过 1 个百分点。
  之后可试：INT4、稀疏版本单独定线

总体讨论

对整条路线的疑问、想调整的地方写在这里。和各阶段下的笔记一起，自动保存到 `notes/讨论记录.md`。

各阶段小时数为零基础下的估算投入。路线内容在 `web/data.js`，改完刷新页面即可；运行 `./make_pdf.sh` 可重新导出 PDF。